

ISTANBUL TECHNICAL UNIVERSITY
FACULTY OF SCIENCE AND LETTERS
DEPARTMENT OF MATHEMATICAL ENGINEERING

Fitness Tracker Application

A Native Android Workout Logging System

MAT 335E – Programming Algorithms
Term Project Report

Student 1: Rahmet Sare Yılmaz
Student 2: Kenan Güzel
Course: MAT 335E – Programming Algorithms
Platform: Android (Java)
Repository: github.com/Klimarnock/proje_app
Date: 21 May 2026

Submitted in partial fulfilment of the requirements for
MAT 335E Programming Algorithms, Spring 2026

Contents

1	Introduction	2
1.1	Project Objectives	2
2	System Design	2
2.1	Architecture Overview	2
2.2	Class Structure	2
3	Database Design	3
3.1	The <code>athletes</code> Table	3
3.2	The <code>activities</code> Table	4
3.3	DatabaseHelper Methods	4
3.4	Database Creation Code (excerpt)	4
4	Object-Oriented Design	5
4.1	Inheritance Hierarchy	5
4.2	OOP Principles Applied	5
4.3	Subclass Code Example	5
5	UML Class Diagram	6
6	Graphical User Interface	7
6.1	Screen Highlights	8
6.2	Input Validation	9
7	Implementation Details	10
7.1	Development Environment	10
7.2	Pre-loaded Sample Data	10
7.3	Database Upgrade Strategy	10
8	Conclusion	10

1 Introduction

This report documents the design, architecture, and implementation of the **Fitness Tracker** application, developed as the term project for MAT 335E Programming Algorithms. The project was built collaboratively by Rahmet Sare Yılmaz and Kenan Güzel.

The Fitness Tracker is a native Android application written in Java using Android Studio. It allows athletes to register personal profiles, log workout sessions categorised by sport type (Running, Cycling, Swimming), and review their fitness history. Persistent storage is handled by a local SQLite database, and the user interacts with the system through seven distinct graphical screens.

The motivation behind the project comes from a personal interest in triathlon training and the practical need for a lightweight mobile tool to track diverse workout metrics in one place.

1.1 Project Objectives

- Develop a mobile-based sports and fitness tracking system using Java and Android.
- Apply object-oriented programming principles: *encapsulation*, *inheritance*, and *polymorphism*.
- Provide a user-friendly graphical interface for quick workout logging.
- Manage athlete profile data and historical workout logs using SQLite.
- Display individual workout history per athlete with summary statistics.
- Provide a global activity feed showing all logged workouts across all athletes.
- Implement input validation and error handling for reliable behaviour.

2 System Design

2.1 Architecture Overview

The application follows a layered Android architecture that cleanly separates concerns:

- **Model Layer:** Plain Java classes (*Athlete*, *Activity*, *Running*, *Cycling*, *Swimming*) representing domain objects.
- **Data Layer:** The *DatabaseHelper* class managing all SQLite CRUD operations via Android's *SQLiteOpenHelper*.
- **View Layer:** Android XML layout files defining the UI for each screen.
- **Controller Layer:** Android *Activity* Java classes (*MainActivity*, *Dashboard*, etc.) connecting the UI to the data layer.

2.2 Class Structure

The application contains the following Java classes in the `com.example.fitnesstracker` package:

Class	Type	Responsibility
Activity	Base class	Defines common fields (<i>type</i> , <i>date</i> , <i>duration</i> , <i>calories</i>) and the <code>getSummary()</code> method shared by all workout types.
Running	Subclass	Extends Activity ; adds a <i>distance</i> field; overrides <code>getSummary()</code> with running-specific output.
Cycling	Subclass	Extends Activity ; adds <i>distance</i> and <i>averageSpeed</i> fields; overrides <code>getSummary()</code> .
Swimming	Subclass	Extends Activity ; adds a <i>laps</i> field; overrides <code>getSummary()</code> with lap count.
Athlete	Entity	Stores athlete profile: name, age, gender, weight, height, primary sport.
DatabaseHelper	DAO	Extends <code>SQLiteOpenHelper</code> ; manages the athletes and activities tables; provides all CRUD methods.
MainActivity	Controller	Android entry point; coordinates navigation between all screens and handles user interactions via DatabaseHelper .

Table 1: Java classes in the Fitness Tracker application.

3 Database Design

The application uses an SQLite database named `fitness_tracker.db` (version 2) with two tables, both managed by the **DatabaseHelper** class which extends `SQLiteOpenHelper`.

3.1 The athletes Table

Column	Data Type	Description
<code>id</code>	INTEGER (PK, AUTO)	Unique identifier for each athlete.
<code>name</code>	TEXT	Full name of the athlete.
<code>age</code>	INTEGER	Age in years.
<code>gender</code>	TEXT	Gender (Male / Female / Other).
<code>weight</code>	REAL	Body weight in kilograms.
<code>height</code>	REAL	Height in centimetres.
<code>primary_sport</code>	TEXT	Preferred sport: Running, Cycling, or Swimming.

Table 2: Schema of the **athletes** table.

3.2 The activities Table

Column	Data Type	Description
id	INTEGER (PK, AUTO)	Unique identifier for each activity record.
athlete_name	TEXT (FK ref)	Name of the athlete who performed the activity.
type	TEXT	Activity type: Running, Cycling, or Swimming.
date	TEXT	Date of the activity in YYYY-MM-DD format.
duration	INTEGER	Duration in minutes.
calories	INTEGER	Estimated calories burned.
distance	REAL	Distance in km (Running/Cycling); 0 for Swimming.
laps	INTEGER	Number of laps (Swimming only); 0 otherwise.
avg_speed	REAL	Average speed in km/h (Cycling only); 0 otherwise.

Table 3: Schema of the `activities` table.

3.3 DatabaseHelper Methods

Method	Returns	Description
<code>insertAthlete(...)</code>	boolean	Inserts a new athlete record; returns false on failure.
<code>insertActivity(...)</code>	boolean	Inserts a new activity record with all fields.
<code>athleteExists(name)</code>	boolean	Checks if an athlete with the given name already exists.
<code>getActivitiesByAthlete(name)</code>	Cursor	Returns all activities for a specific athlete, newest first.
<code>getTotalWorkoutCount(name)</code>	int	Returns total workout count for an athlete.
<code>getTotalCalories(name)</code>	int	Returns sum of all calories burned.
<code>getTotalDuration(name)</code>	int	Returns total minutes spent working out.
<code>deleteActivitiesByAthlete(name)</code>	void	Deletes all activity records for an athlete.
<code>getGlobalActivityFeed()</code>	Cursor	Returns all activities across all athletes, newest first.

Table 4: Public methods exposed by `DatabaseHelper`.

3.4 Database Creation Code (excerpt)

```

1 @Override
2 public void onCreate(SQLiteDatabase db) {
3
4     String createAthleteTable =
5         "CREATE TABLE athletes (" +
6         "id INTEGER PRIMARY KEY AUTOINCREMENT, " +

```

```

7      "name TEXT, age INTEGER, gender TEXT, " +
8      "weight REAL, height REAL, primary_sport TEXT)";
9
10     String createActivityTable =
11         "CREATE TABLE activities (" +
12         "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
13         "athlete_name TEXT, type TEXT, date TEXT, " +
14         "duration INTEGER, calories INTEGER, " +
15         "distance REAL, laps INTEGER, avg_speed REAL)";
16
17     db.execSQL(createAthleteTable);
18     db.execSQL(createActivityTable);
19 }

```

Listing 1: DatabaseHelper.java – table creation in onCreate().

4 Object-Oriented Design

4.1 Inheritance Hierarchy

The `Activity` class serves as the base class for all workout types. Each subclass inherits common fields and overrides `getSummary()` to produce sport-specific formatted output.

Class	Own Fields	getSummary() Output
Activity	<i>(base)</i>	Generic: <i>type – date – duration min – calories kcal</i>
Running	distance (km)	Running date X min Y km Z kcal
Cycling	distance, averageSpeed	Cycling date X min Y km Z km/h W kcal
Swimming	laps (count)	Swimming date X min Y laps Z kcal

Table 5: Inheritance hierarchy and per-subclass output format.

4.2 OOP Principles Applied

- **Inheritance:** `Running`, `Cycling`, and `Swimming` all extend `Activity`, using `super()` to reuse common constructor logic and inherited fields.
- **Polymorphism:** `getSummary()` is overridden in each subclass, producing sport-specific output and demonstrating runtime polymorphism.
- **Encapsulation:** All class fields are declared `private` or `protected`, exposed only via constructors and getter methods.
- **Abstraction:** The `Activity` class abstracts common workout behaviour, hiding sport-specific details inside each subclass.

4.3 Subclass Code Example

```

1 public class Cycling extends Activity {
2
3     private double distance;

```

```
4     private double averageSpeed;
5
6     public Cycling(String date, int duration, int calories,
7                     double distance, double averageSpeed) {
8         super("Cycling", date, duration, calories);
9         this.distance = distance;
10        this.averageSpeed = averageSpeed;
11    }
12
13    @Override
14    public String getSummary() {
15        return "Cycling | " + date +
16               " | " + duration + " min | " +
17               distance + " km | " +
18               averageSpeed + " km/h | " +
19               calories + " kcal";
20    }
21 }
```

Listing 2: Cycling.java – subclass with overridden getSummary().

5 UML Class Diagram

The UML class diagram in Figure 1 illustrates all classes, their fields and methods, and the relationships between them.

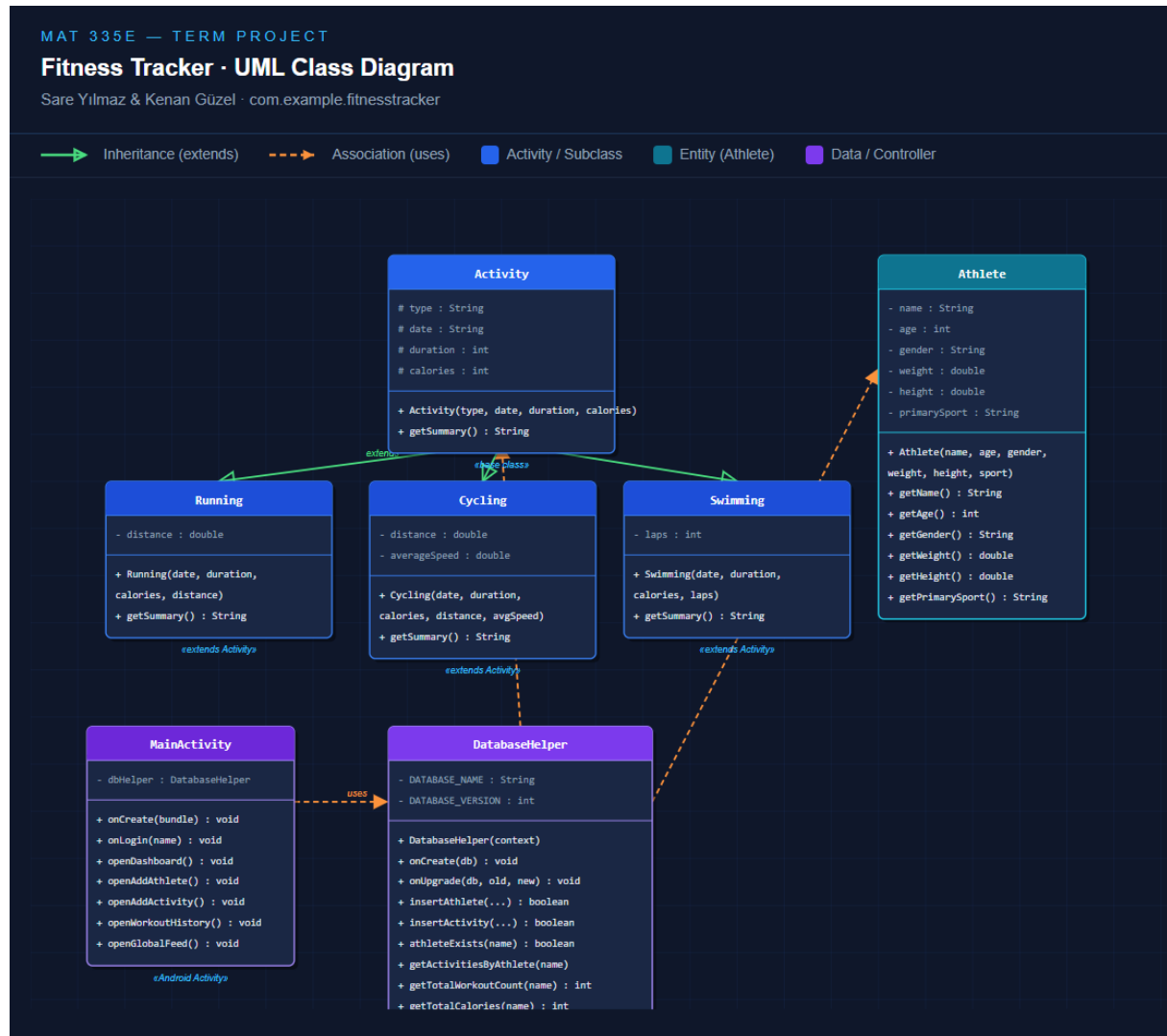


Figure 1: UML class diagram of the Fitness Tracker application.

Relationships shown in the diagram:

- **Inheritance (extends):** Running, Cycling, and Swimming each extend Activity.
- **Association (uses):** DatabaseHelper manages both Athlete and Activity records.
- **Association (uses):** MainActivity uses DatabaseHelper for all data operations.
- **Dependency:** DatabaseHelper extends Android's SQLiteOpenHelper.

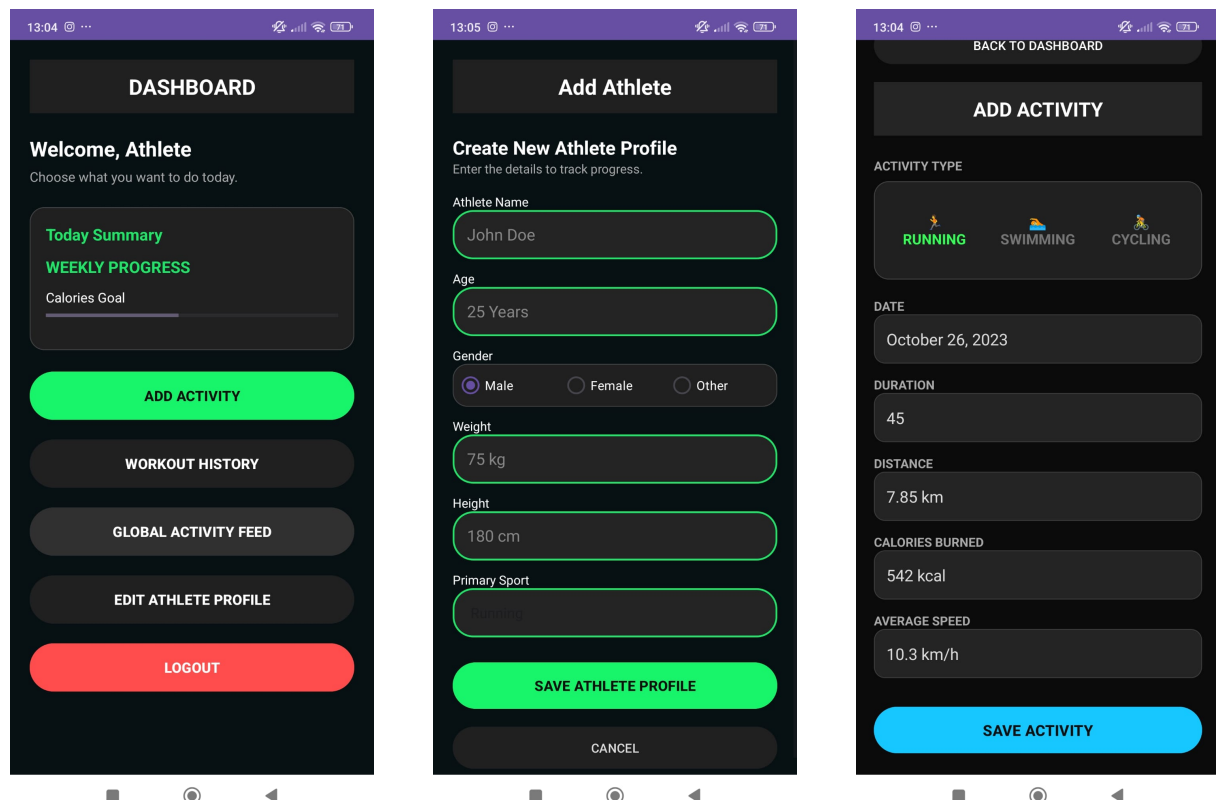
6 Graphical User Interface

The application consists of seven Android Activity screens, each with a corresponding XML layout file.

Screen	Layout File	Description
Login / Welcome	activity_login.xml	Entry screen; user selects or enters an athlete name to begin a session.
Main Menu	activity_main.xml	Central navigation hub with buttons to all other screens.
Dashboard	activity_dashboard.xml	Displays statistics: total workouts, calories, duration.
Add Athlete	activity_add_athlete.xml	Form to register a new athlete; validates against duplicate names.
Add Activity	activity_add_activity.xml	Form to log a workout; dynamically shows fields per selected sport.
Workout History	activity_workout_history.xml	Lists all past activities for the logged-in athlete, newest first.
Global Feed	activity_global_feed.xml	Shows all activities across all athletes as a community feed.

Table 6: The seven Activity screens of the Fitness Tracker application.

6.1 Screen Highlights

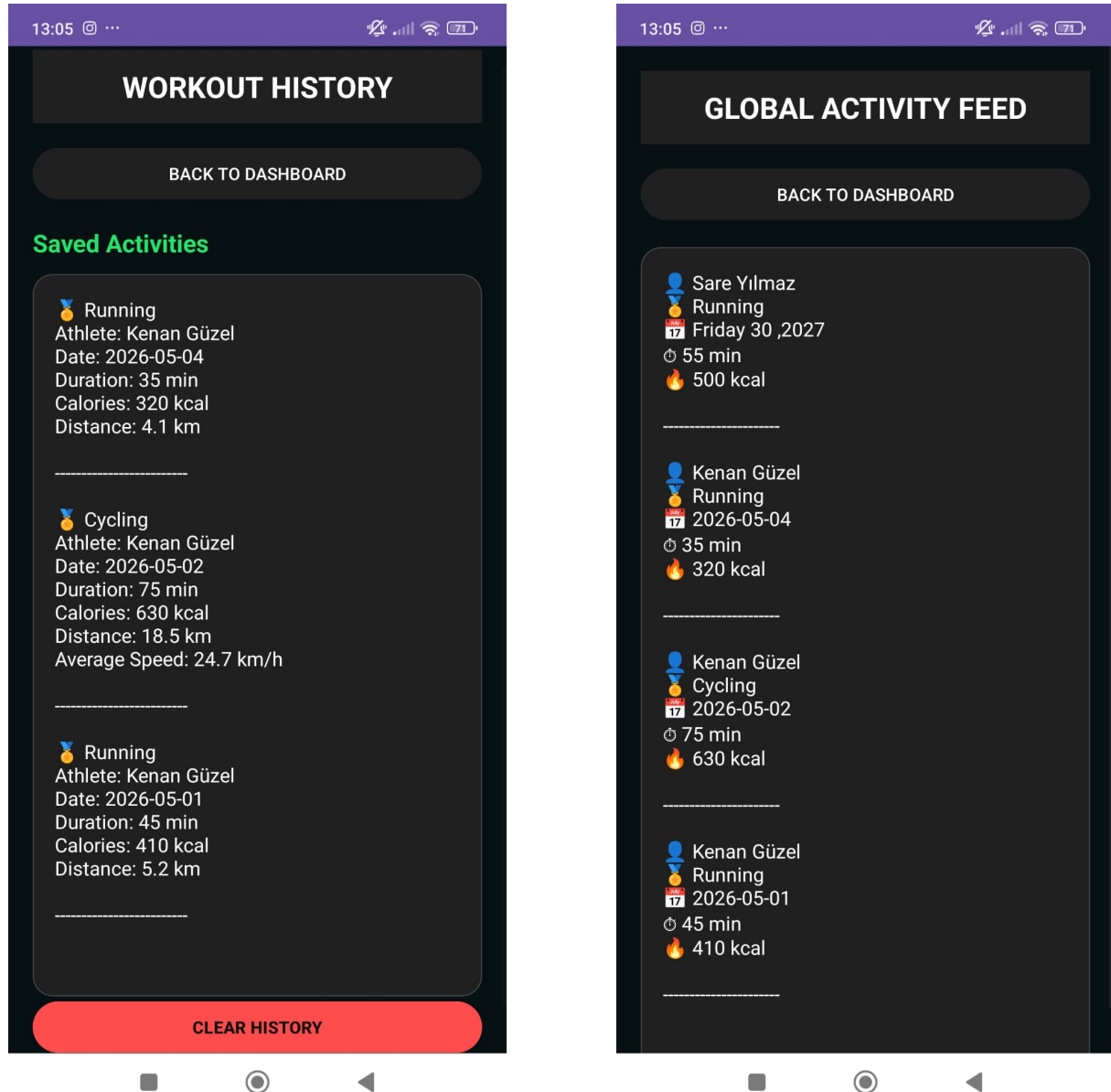


(a) Dashboard with weekly progress.

(b) Add Athlete profile form.

(c) Add Activity (Running selected).

Figure 2: Athlete-facing screens: dashboard, registration, and workout logging.



(a) Personal Workout History screen.

(b) Global Activity Feed screen.

Figure 3: History and community-feed views, both backed by SQLite queries.

6.2 Input Validation

The application performs validation across all input forms:

- **Duplicate athlete detection:** `athleteExists()` is called before inserting a new athlete to prevent duplicate entries.
- **Required field checks:** all mandatory form fields are validated before any database write.
- **Dynamic field display:** the Add Activity screen shows or hides fields (distance, laps, speed) based on the selected activity type, as visible in Figure 2c.
- **User feedback:** Toast messages and dialog alerts inform the user of both validation errors and successful operations.

7 Implementation Details

7.1 Development Environment

Item	Detail
IDE	Android Studio
Language	Java
Database	SQLite via <code>android.database.sqlite.SQLiteOpenHelper</code>
Target Platform	Android mobile devices
Version Control	Git / GitHub (github.com/Klimarnock/proje_app)

Table 7: Development environment summary.

7.2 Pre-loaded Sample Data

The `DatabaseHelper.onCreate()` method seeds the database with sample data so that the application is functional from first launch:

- **Athletes:** Kenan Güzel (Running), Sare Yılmaz (Swimming), Alex Carter (Running), Emma Johnson (Swimming), Michael Brown (Cycling).
- **Activities:** six pre-seeded workout records covering Running, Cycling, and Swimming sessions in May 2026.

7.3 Database Upgrade Strategy

The `onUpgrade()` method drops and recreates both tables when the database version is incremented, ensuring schema changes are cleanly applied during development iterations.

8 Conclusion

The Fitness Tracker application successfully fulfils all requirements set out in the MAT 335E term assignment.

Requirement	Status	Implementation
UML Class Diagram		Full class hierarchy with members, methods, and relationships.
GUI Application		7 interactive Android screens with navigation and form validation.
Database Integration		SQLite with 2 tables and 9 CRUD operations in <code>DatabaseHelper</code> .
Input Validation		Duplicate checks, required fields, sport-specific dynamic forms.
OOP Design		Inheritance, polymorphism, encapsulation across the <code>Activity</code> hierarchy.
Code–UML Consistency		All UML classes directly correspond to source code files.

Table 8: Requirement compliance matrix.

The project demonstrates practical application of object-oriented programming alongside Android development, SQLite database management, and GUI design. The codebase is organised, maintainable, and ready for presentation.

References

- [1] Android Developer Documentation – *SQLiteOpenHelper*.
[developer.android.com] (<https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper>)
- [2] Android Developer Documentation – *Activities and Intents*.
[developer.android.com] (<https://developer.android.com/guide/components/activities>)
- [3] Oracle Java Documentation – *Inheritance and Polymorphism*.
[docs.oracle.com] (<https://docs.oracle.com/javase/tutorial/java/IandI>)
- [4] Project Source Code Repository.
[github.com] (https://github.com/Klimarnock/proje_app)